Introduction to Deep Learning

Session 9: Convolutional Neural Networks

May 2026

Magda Gregorová - magda.gregorova@thws.de

thws
Technical University
of Applied Sciences
Würzburg-Schweinfurt

Lecture Overview

thws
Technical University
of Applied Sciences
Würzburg-Schweinfurt

1. Why Not MLPs for Images?
2. Convolution Operation
3. Pooling
4. CNN Architecture
5. Classic Architectures
6. Modern Details
7. Summary

Images as Data

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Image: tensor of shape $H \times W \times C$

- H, W — height and width in pixels
- C — channels (1 for grayscale, 3 for RGB)

Typical sizes:

Dataset	Size	Input dimensions
MNIST	$28 \times 28 \times 1$	784
CIFAR-10	$32 \times 32 \times 3$	3 072
ImageNet	$224 \times 224 \times 3$	150 528

MLP approach: flatten image to vector $\mathbf{x} \in \mathbb{R}^{H \cdot W \cdot C}$, apply fully connected layers

This leads to fundamental problems

Parameter Explosion

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

MLP on ImageNet images: input dimension = $224 \times 224 \times 3 = 150\,528$

First hidden layer with 1024 units:

$$150\,528 \times 1024 \approx 154 \text{ million parameters} \quad \text{(one layer only)}$$

For comparison:

- ResNet-50 (entire network): ~25 million parameters
- VGG-16 (entire network): ~138 million parameters

Consequences:

- Enormous memory and compute requirements
- Massive overfitting — far more parameters than training examples
- Impractical to train even with modern hardware

No Spatial Structure

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

MLP treats each pixel independently — no notion of spatial neighbourhood

Problem 1: no translation invariance

- Cat in top-left corner activates completely different weights than cat in bottom-right
- Same concept must be learned independently at every location
- $H \times W$ times more parameters needed

Problem 2: no local structure

- Nearby pixels are highly correlated — edges, textures, shapes span local regions
- MLP ignores this — pixel $(0, 0)$ and pixel $(223, 223)$ treated identically
- Spatial relationships not exploited

We need an architecture that exploits the spatial structure of images

What We Need

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Three properties that an image-aware architecture should have:

- **Local connectivity** — each neuron connects only to a small local region of the input (the **receptive field**), not the entire image
- **Parameter sharing** — the same filter (weights) applied at every spatial location — detects the same feature everywhere
- **Translation invariance** — detecting a feature regardless of where it appears in the image

These three properties define the **convolutional layer**

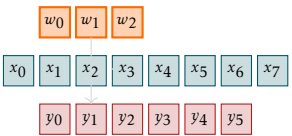
1D Convolution — Intuition

1D convolution: filter $\mathbf{w} \in \mathbb{R}^k$ slides over signal $\mathbf{x} \in \mathbb{R}^n$

$$(x * w)[i] = \sum_{j=0}^{k-1} x[i+j]\, w[j]$$

- At each position i: dot product between filter and local patch of signal
- Filter detects a specific local pattern
- Same filter applied at every position — **parameter sharing**

Example: edge detector, smoothing filter, derivative approximation

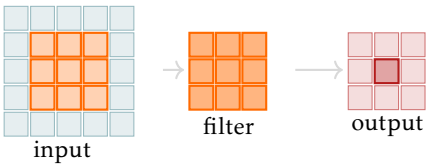


2D Convolution

2D convolution: filter $\mathbf{W} \in \mathbb{R}^{k \times k}$ slides over image $\mathbf{X} \in \mathbb{R}^{H \times W}$

$$(\mathbf{X} * \mathbf{W})[i, j] = \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} \mathbf{X}[i+p, j+q]\, \mathbf{W}[p, q]$$

- At each position (i, j): dot product between filter and $k \times k$ patch
- Output: **feature map** of shape $(H - k + 1) \times (W - k + 1)$
- Filter learns to detect a specific visual pattern (edge, corner, texture)



Stride and Padding

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Stride s: filter moves s pixels at each step

- $s = 1$: standard, output size $\approx$ input size
- $s = 2$: output halved — spatial downsampling
- Reduces computation and memory

Output size: $\lfloor (H - k)/s \rfloor + 1$

Padding p: add zeros around the border

- **Valid** ($p = 0$): no padding, output smaller than input
- **Same** ($p = (k-1)/2$): output same size as input
- Preserves spatial dimensions

Output size: $\lfloor (H + 2p - k)/s \rfloor + 1$

Multiple Channels and Filters

thws
Technical University
of Applied Sciences
Würzburg-Schweinfurt

RGB input: C_{in} input channels — filter must have depth C_{in}

$$(\mathbf{X} * \mathbf{W})[i, j] = \sum_{c=0}^{C_{\text{in}}-1} \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} \mathbf{X}[c, i+p, j+q]\, \mathbf{W}[c, p, q]$$

Multiple filters: C_{out} filters produce C_{out} output feature maps

- Each filter detects a different pattern
- Output tensor: $C_{\text{out}} \times H' \times W'$
- Total parameters per layer: $C_{\text{out}} \times C_{\text{in}} \times k \times k + C_{\text{out}}$ (with bias)

In PyTorch:

```
nn.Conv2d(in_channels=3, out_channels=64, kernel_size=3, stride=1, padding=1)
```

Receptive Field

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Receptive field: region of the input that influences a given neuron's output

- Layer 1 (3×3 filter): receptive field = 3×3
- Layer 2 (3×3 filter): receptive field = 5×5
- Layer L (3×3 filters): receptive field = $(2L+1) \times (2L+1)$

Key insight: deep networks have large receptive fields despite small local filters

- Early layers detect local features (edges, corners)
- Later layers combine local features into complex patterns (eyes, wheels, faces)
- Hierarchical feature learning — the hallmark of deep networks

Two 3×3 filters have the same receptive field as one 5×5 filter but fewer parameters: $2 \times 9 = 18$ vs 25

Pooling

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Pooling: spatial downsampling — reduces feature map size

Max pooling:

$$y[i,j] = \max_{p,q \in \mathcal{R}_{ij}} x[p,q]$$

- Takes maximum over local region
- Retains strongest activation
- Most commonly used

Average pooling:

$$y[i,j] = \frac{1}{|\mathcal{R}_{ij}|} \sum_{p,q \in \mathcal{R}_{ij}} x[p,q]$$

- Takes average over local region
- Smoother downsampling
- Less common in hidden layers

Benefits:

- Reduces spatial dimensions — fewer parameters in subsequent layers
- Local translation invariance — small shifts in feature position do not affect output

Global Average Pooling

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Global average pooling (GAP): averages entire feature map to a single value per channel

$$y_c = \frac{1}{H \cdot W} \sum_{i=1}^{H} \sum_{j=1}^{W} x_c[i, j]$$

- Output: vector of length C_{out} — one value per feature map
- Replaces large FC layers at the end of the network
- Dramatically reduces parameters
- Forces each feature map to represent a single concept

Used in: ResNet, GoogLeNet, MobileNet — modern standard

In PyTorch: `nn.AdaptiveAvgPool2d((1,1))` followed by `flatten`

CNN Building Block

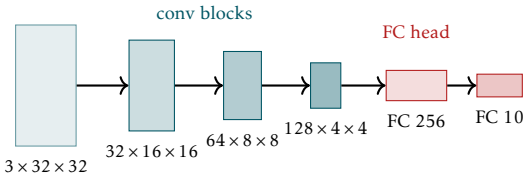
thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Standard pattern: Conv → BatchNorm → ReLU → (Pool)

- **Conv:** extract features via learned filters
- **BatchNorm:** stabilise activations — essential for deep CNNs
- **ReLU:** non-linearity — enables complex feature combinations
- **Pool:** spatial downsampling — reduce resolution, increase receptive field

Full CNN Architecture

Two stages: feature extraction (conv blocks) → classification (FC head)



Conv blocks:

- Increasing channels: $3 \to 32 \to 64 \to 128$
- Decreasing spatial size via pooling
- Learn hierarchical features

FC head:

- Flatten spatial features
- One or more FC layers
- Final softmax for classification

LeNet-5 (LeCun et al., 1998)

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

First successful CNN — handwritten digit recognition (MNIST)

- 2 conv layers + 2 FC layers
- 28×28 input $\rightarrow$ 10 class output
- ~60k parameters — tiny by modern standards
- Used sigmoid/tanh activations, average pooling

Significance:

- Demonstrated that learned features outperform hand-crafted ones
- Introduced the conv $\rightarrow$ pool $\rightarrow$ conv $\rightarrow$ pool $\rightarrow$ FC pattern
- Deployed commercially for cheque reading

Limitation: too shallow and small for complex tasks — needed more compute

AlexNet (Krizhevsky et al., 2012)

thws
Technical University
of Applied Sciences
Würzburg-Schweinfurt

ImageNet moment — won ILSVRC 2012 with 15.3% top-5 error vs 26.2% runner-up

- 5 conv layers + 3 FC layers, ~60M parameters
- Key innovations:
  - ReLU activations instead of sigmoid/tanh — faster training
  - Dropout in FC layers — regularisation
  - Data augmentation — random crops, horizontal flips
  - GPU training — two GTX 580 GPUs
  - Local response normalisation (since superseded by BatchNorm)

Impact: launched the modern deep learning era — proved deep CNNs work at scale

VGG (Simonyan & Zisserman, 2014)

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Simplicity through depth — all 3×3 convolutions, increasing depth

- VGG-16: 13 conv layers + 3 FC layers, ~138M parameters
- Only 3×3 filters throughout — simplicity as a design principle
- Depth: 16–19 layers

Key insight: two 3×3 conv layers have same receptive field as one 5×5 but:

- Fewer parameters: $2 \times 3^2 = 18$ vs $5^2 = 25$
- More non-linearities — more expressive

Limitation: 3 large FC layers contain most parameters (~100M) — memory intensive

Legacy: VGG features widely used for transfer learning; clean design still instructive

ResNet (He et al., 2016)

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Problem: deeper networks were performing worse than shallower ones

Degradation problem: adding more layers hurts training accuracy — not overfitting, but optimisation failure

Key idea: **residual connections** (skip connections)

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{\mathbf{W}_i\}) + \mathbf{x}$$

- Instead of learning $\mathbf{y} = \mathcal{F}(\mathbf{x})$, learn the **residual** $\mathcal{F}(\mathbf{x}) = \mathbf{y} - \mathbf{x}$
- If optimal solution is identity, easier to learn $\mathcal{F}(\mathbf{x}) = 0$ than $\mathcal{F}(\mathbf{x}) = \mathbf{x}$
- Gradient flows directly through skip connection — no vanishing gradients

Result: successfully trained networks with 50, 101, 152 layers

Residual Block

Basic residual block:

$$\mathbf{y} = \text{ReLU}(\mathcal{F}(\mathbf{x}) + \mathbf{x})$$

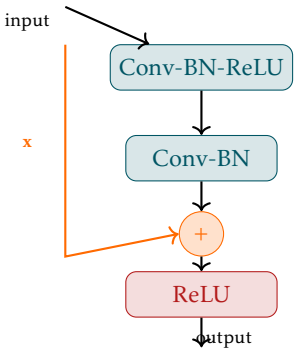
$\mathcal{F}(\mathbf{x})$: two Conv-BN-ReLU layers

Bottleneck block (ResNet-50+):

- 1×1 conv: reduce channels
- 3×3 conv: spatial processing
- 1×1 conv: restore channels

Fewer parameters than basic block at same depth

Projection shortcut: when dimensions change, use 1×1 conv on skip connection to match



1×1 Convolutions

1×1 **conv:** applies a learned linear combination across channels at each spatial location

$$y_c[i,j] = \sum_{c'=1}^{C_{\text{in}}} w_{c,c'}\, x_{c'}[i,j] + b_c$$

Equivalent to: FC layer applied independently at each spatial location

Uses:

- **Channel reduction/expansion** — change C_{in} to C_{out} cheaply (bottleneck blocks)
- **Non-linearity injection** — add ReLU between channels without spatial processing
- **Network-in-network** — increase model capacity without larger filters
- **Projection shortcuts** — match dimensions in residual connections

Depthwise Separable Convolutions

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Standard conv: $C_{\text{out}} \times C_{\text{in}} \times k \times k$ parameters

Depthwise separable conv: split into two steps

Depthwise conv:

- One filter per input channel
- Spatial processing only
- $C_{\text{in}} \times k \times k$ parameters

Pointwise conv (1×1)**:**

- Mix channels
- $C_{\text{out}} \times C_{\text{in}}$ parameters

Parameter reduction: factor of $\approx k^2$ — for 3×3: $\sim 9\times$ fewer parameters

Used in: MobileNet, EfficientNet — efficient architectures for mobile/edge deployment

Batch Norm in CNNs

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

BatchNorm2d: normalises over batch **and** spatial dimensions for each channel

$$\mu_c = \frac{1}{N \cdot H \cdot W} \sum_{n,i,j} x_c^{(n)}[i,j], \qquad \sigma_c^2 = \frac{1}{N \cdot H \cdot W} \sum_{n,i,j} (x_c^{(n)}[i,j] - \mu_c)^2$$

- One mean and variance per **channel** — not per spatial location
- Learnable γ_c and β_c per channel

Placement: Conv → BN → ReLU (standard) or Conv → ReLU → BN (less common)

Effect in CNNs:

- Essential for training deep CNNs — without it, networks beyond ~10 layers are very hard to train
- Allows higher learning rates
- Reduces need for careful initialisation

Dropout in CNNs

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Standard dropout on FC layers works well in CNNs too

Problem with dropout on conv layers:

- Adjacent pixels are highly correlated
- Dropping individual activations does not create enough independence
- Information easily propagated through neighbouring non-dropped activations

SpatialDropout2d: drop entire feature maps (channels) rather than individual activations

- Removes all spatial locations of a channel simultaneously
- Forces network to use diverse feature maps
- More effective regularisation for conv layers

In practice: BatchNorm often sufficient regularisation in conv layers — dropout mainly used in FC head

Summary

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Why CNNs:

- Local connectivity
- Parameter sharing
- Translation invariance

Building blocks:

- Conv: feature extraction
- BatchNorm + ReLU: stability + non-linearity
- Pooling / stride: downsampling
- FC head: classification

Key architectures:

- LeNet → AlexNet → VGG → ResNet
- Residual connections: key to deep networks

Modern details:

- 1×1 conv: channel mixing
- Depthwise separable: efficiency
- BatchNorm2d: per-channel normalisation
- SpatialDropout: conv regularisation

In PyTorch:

- `nn.Conv2d`
- `nn.MaxPool2d`
- `nn.BatchNorm2d`
- `nn.AdaptiveAvgPool2d`

Next: L10 — Sequence Models & RNNs